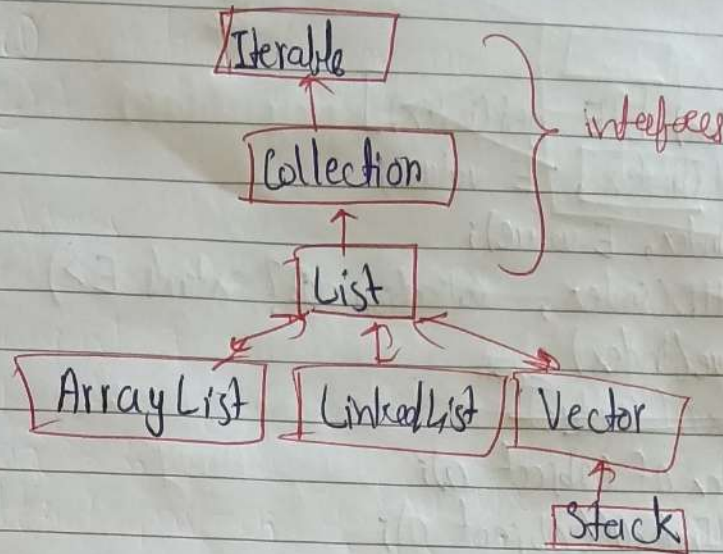
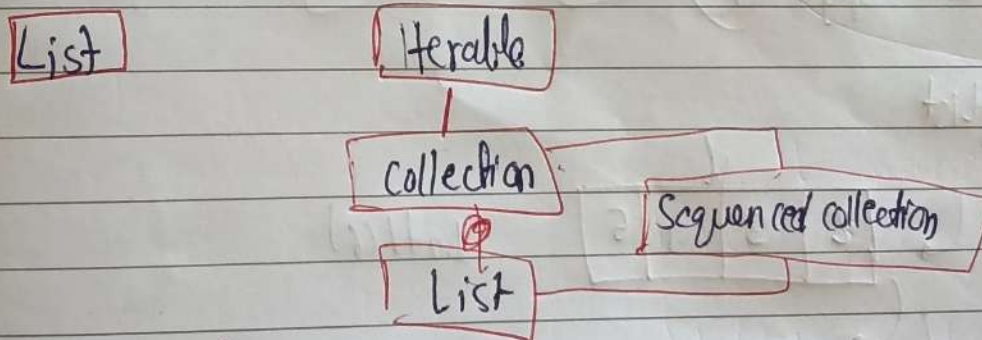


List Interface



* List

- elements have a position
- Duplicates are allowed
- Insertion order is preserved.
- you can access element by index



Sequenced Collection

- getFirst()
- getLast()
- addFirst()
- addLast()
- removeFirst()
- removeLast()

List

- get(index) [E get(index)]
- set(index, E value);
- add(index, E value);
- addAll(index, Collection < E ? extends E >)
- remove(index)
- ~~indexOf(value)~~
- index of (Object o);
- lastIndexOf (Object o);
- listIterator();
- listIterator(int index);
- Of (-);
- copyOf ();
- ↳ List<Integer> l = List.of(1, 2, 3); //immutable

* ArrayList

get(index) → O(N)

set(i)

add(i, o) → O(N)

remove(i, o)

↳

2	3	4	5
---	---	---	---

 d.add(6)

↳

2	3	4	5	6		
---	---	---	---	---	--	--

 ✗ (not reality) (2x ✗)

↳

$$\text{new capacity} = \text{old capacity} + \left(\frac{\text{old capacity}}{2} \right)$$

 (in reality) (1.5x ✓)

ArrayList $\rightarrow$ Iterable, collection, sequential collection, List
 (90% usecase) in real world

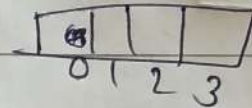
- Random access
- Cache Friendly
- Simple structure

Constructors

List<Integer> l = new ArrayList<>();
 = new ArrayList<>(10);
 = new ArrayList<>(collection);

ArrayList

- ensureCapacity();
- trimToSize();



l.ensureCapacity(100);
 l.trimToSize();

Capacity
 size

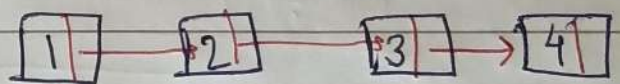
* LinkedList (10% usecase) in real world

```
class Node<T> {
    T data;
    Node<T> next;
    Node<T> previous;
}
```

l.add(1)
 l.add(2)

add()
 remove()
 addAll()
 set()

$\rightarrow O(N)$



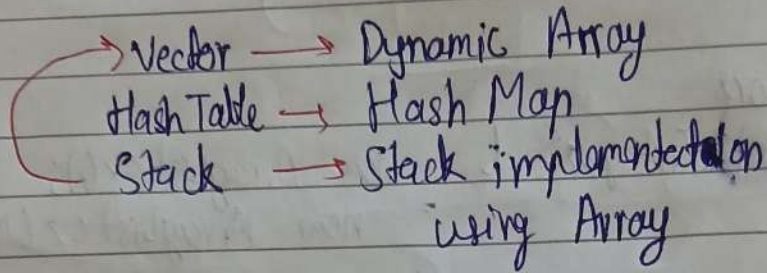
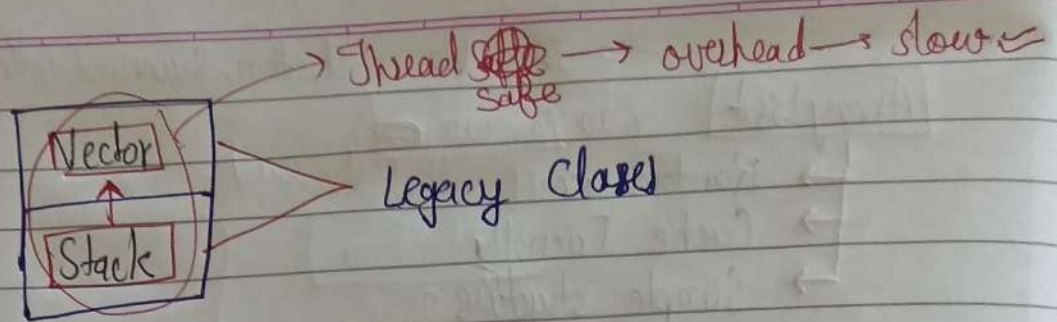
Single Linked List



doubly Linked List

list.get(2);
 if (index < size/2)
 start from first

Queue / Stack



List

Queue

Set

ArrayDeque

Queue

Stack